

Student Performance Analysis using EDA

Detailed Exploratory Data Analysis (EDA) Project Guide

1. Project Introduction

EDA (Exploratory Data Analysis) means exploring a dataset to understand its structure, patterns, trends, distributions, relationships, and unusual observations. In this project, student academic and related information is analyzed to understand factors associated with student performance. The project focuses on data exploration rather than machine-learning prediction.

2. Problem Statement

Educational institutions can collect large amounts of student information, but raw rows of data are difficult to interpret directly. This project transforms raw student information into meaningful statistics, visualizations, relationships, and documented insights.

Problem: Analyze student-related data and identify patterns, trends, distributions, and relationships that can help understand academic performance.

3. Project Objectives

- Understand the dataset structure and variables.
- Identify and handle missing values and duplicate records.
- Calculate descriptive statistical summaries.
- Analyze distributions and potential outliers.
- Create useful visualizations using Matplotlib and Seaborn.
- Study relationships between study hours, attendance, assessments, and final marks.
- Use correlation analysis to identify linear associations.
- Communicate findings in a structured EDA report.

4. Technologies Used

Technology	Purpose
Python	Main programming language
Pandas	Data loading, cleaning, and manipulation
NumPy	Numerical operations
Matplotlib	Data visualization
Seaborn	Statistical and advanced visualization
Jupyter Notebook	Interactive analysis and documentation
GitHub	Project hosting and portfolio presentation

5. Dataset

Column	Meaning
Student_ID	Unique student identifier
Gender	Student gender category
Age	Student age
Study_Hours	Daily study hours
Attendance	Attendance percentage
Assignment_Score	Assignment marks
Internal_Marks	Internal examination marks
Lab_Score	Laboratory marks
Previous_Performance	Previous academic score
Final_Marks	Final examination marks

6. Step 1 — Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

These libraries provide the main tools required for data manipulation, numerical calculations, and visualization.

7. Step 2 — Load Dataset

```
df = pd.read_csv("student_performance.csv")
df.head()
```

The CSV file is loaded into a Pandas DataFrame. `head()` displays the first five records.

8. Step 3 — Understand Dataset Structure

```
print(df.shape)
df.info()
df.head()
df.tail()
df.sample(5)
```

`shape` shows rows and columns, while `info()` shows column names, non-null counts, data types, and memory information.

9. Step 4 — Statistical Summary

```
df.describe()
```

This produces descriptive statistics such as count, mean, standard deviation, minimum, quartiles, and maximum for numerical columns.

10. Step 5 — Check Missing Values

```
df.isnull().sum()
```

This identifies columns containing missing observations. Missing values should be handled using an approach appropriate to the dataset.

11. Step 6 — Handle Missing Values

```
df["Study_Hours"] = df["Study_Hours"].fillna(
    df["Study_Hours"].median()
)

df["Gender"] = df["Gender"].fillna(
    df["Gender"].mode()[0]
)
```

Numerical missing values can sometimes be replaced using the median, while categorical values can sometimes use the mode. The chosen method should be justified.

12. Step 7 — Check Duplicate Records

```
print(df.duplicated().sum())
df = df.drop_duplicates()
print(df.duplicated().sum())
```

Duplicate rows can distort statistics and should be investigated and removed when they represent repeated records.

13. Step 8 — Check Data Types

```
df.dtypes
```

This verifies whether each variable has an appropriate data type.

14. Step 9 — Gender Distribution

```
print(df["Gender"].value_counts())

sns.countplot(data=df, x="Gender")
plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Number of Students")
plt.show()
```

A count plot shows the number of students in each gender category and helps us understand the categorical distribution.

15. Step 10 — Final Marks Distribution

```
plt.figure(figsize=(8,5))
sns.histplot(df["Final_Marks"], bins=10, kde=True)
plt.title("Distribution of Final Marks")
plt.xlabel("Final Marks")
plt.ylabel("Number of Students")
plt.show()
```

The histogram helps identify where student scores are concentrated and whether the distribution appears symmetric or skewed.

16. Step 11 — Study Hours vs Final Marks

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="Study_Hours", y="Final_Marks")
plt.title("Study Hours vs Final Marks")
plt.xlabel("Study Hours")
plt.ylabel("Final Marks")
plt.show()
```

Each point represents a student. The plot allows us to visually inspect whether study hours and final marks have a positive, negative, or weak linear association. A relationship does not by itself prove causation.

17. Step 12 — Attendance vs Final Marks

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="Attendance", y="Final_Marks")
plt.title("Attendance vs Final Marks")
plt.xlabel("Attendance (%)")
plt.ylabel("Final Marks")
plt.show()
```

18. Step 13 — Assignment Score vs Final Marks

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="Assignment_Score", y="Final_Marks")
plt.title("Assignment Score vs Final Marks")
plt.xlabel("Assignment Score")
plt.ylabel("Final Marks")
plt.show()
```

19. Step 14 — Box Plot and Outliers

```
plt.figure(figsize=(8,5))
sns.boxplot(y=df["Final_Marks"])
plt.title("Final Marks Box Plot")
plt.ylabel("Final Marks")
plt.show()
```

A box plot summarizes the median, quartiles, spread, and potential outliers. Unusual observations should be investigated rather than automatically deleted.

20. Step 15 — Correlation Analysis

```
correlation = df.corr(numeric_only=True)
print(correlation)

correlation["Final_Marks"].sort_values(ascending=False)
```

Correlation measures the strength and direction of linear association between numerical variables. Values close to +1 indicate strong positive linear association, values close to -1 indicate strong negative linear association, and values near 0 indicate little linear association. Correlation does not establish causation.

21. Step 16 — Correlation Heatmap

```
plt.figure(figsize=(10,7))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
```

```
plt.title("Correlation Heatmap")  
plt.show()
```

The heatmap provides a visual overview of correlations among numerical variables and can help identify relationships worth investigating further.

22. Step 17 — Performance Categories

```
def performance_category(mark):  
    if mark >= 90:  
        return "Excellent"  
    elif mark >= 75:  
        return "Good"  
    elif mark >= 60:  
        return "Average"  
    else:  
        return "Needs Improvement"  
  
df["Performance_Category"] = df["Final_Marks"].apply(  
    performance_category  
)  
  
sns.countplot(data=df, x="Performance_Category")  
plt.title("Student Performance Categories")  
plt.show()
```

This creates simple performance groups from final marks. The thresholds are project-defined and should be documented clearly.

23. Final EDA Workflow

1. Raw Dataset
2. Load CSV File
3. Understand Dataset
4. Data Cleaning
5. Statistical Analysis
6. Data Visualization
7. Correlation Analysis
8. Pattern Identification
9. Generate Insights
10. Final EDA Report
11. GitHub Repository

24. Key Insights to Report

- Describe the overall distribution of final marks.
- Explain the observed relationship between study hours and final marks.
- Explain the observed relationship between attendance and final marks.
- Identify numerical variables with relatively stronger linear associations with final marks.
- Mention potential outliers and how they were investigated.
- Clearly distinguish observed associations from causal claims.

25. GitHub Repository Structure

```
Student-Performance-EDA/
├──
├── dataset/
│   └── student_performance.csv
├──
├── notebooks/
│   └── Student_Performance_EDA.ipynb
├──
├── visualizations/
│   ├── final_marks_distribution.png
│   ├── gender_distribution.png
│   ├── study_hours_vs_marks.png
│   ├── attendance_vs_marks.png
│   ├── assignment_vs_marks.png
│   └── correlation_heatmap.png
├──
├── report/
│   └── EDA_Report.pdf
├──
├── README.md
├── requirements.txt
└── LICENSE
```

26. requirements.txt

```
pandas
numpy
matplotlib
seaborn
jupyter
```

27. How to Run the Project

- Clone the repository: `git clone https://github.com/your-username/Student-Performance-EDA.git`
- Open the project folder.
- Install dependencies using: `pip install -r requirements.txt`
- Start Jupyter Notebook using: `jupyter notebook`
- Open notebooks/Student_Performance_EDA.ipynb and run the cells from top to bottom.

28. Conclusion

This project demonstrates a complete Exploratory Data Analysis workflow. Starting with raw student data, the analysis cleans and examines the dataset, calculates descriptive statistics, creates visualizations, studies correlations, investigates patterns, and communicates findings. The project provides practical experience with Python-based data analysis and creates a professional GitHub portfolio project.

29. Recommended GitHub Presentation

For a strong GitHub submission, include the CSV dataset or a documented data source, the complete Jupyter Notebook, saved visualizations, this PDF report, a clear README, requirements.txt, and an appropriate license. The README should briefly explain the objective, tools, analysis performed, major findings, project structure, and run instructions.